

Analisi delle Prestazioni di Inserimento in Alberi Binari di Ricerca

Londero Stefano
172585

Grasso Federico
173731

Algoritmi e Strutture Dati - 2026

Introduzione

L'obiettivo del progetto è confrontare tra Alberi Binari di Ricerca (BST), Alberi (AVL) e Red-Black (RB) il tempo medio di inserimento di una chiave quando l'albero contiene n chiavi e, ottenere per diversi valori di n , stime robuste dei tempi di esecuzione.

Richiami Teorici

Il costo dell'inserimento in un albero di ricerca dipende dalla lunghezza del cammino radice-foglia, ed è quindi proporzionale alla sua altezza h .

- **BST Semplice:** Non garantisce alcun bilanciamento. Nel caso peggiore può degenerare in $\mathcal{O}(n)$, portando a tempi di inserimento lineari.
- **Alberi AVL:** Mantengono un bilanciamento stretto garantendo che la differenza di altezza tra i sotto-alberi di ogni nodo sia al massimo 1. Nel caso peggiore vengono assicurati tempi asintotici $\mathcal{O}(\log n)$ al costo di frequenti rotazioni strutturali.
- **Alberi Red-Black:** Utilizzando colorazioni per i nodi viene garantita una complessità $\Theta(\log n)$. Sebbene il cammino massimo possa essere leggermente più lungo rispetto a un AVL, il RBT richiede tipicamente meno operazioni di rotazione per ripristinare le proprietà dopo un inserimento.

Metodologia

Per garantire misurazioni robuste e stabili, il progetto si basa su due pilastri:

Varietà Morfologica

Invece di

1. Selezionare una chiave non ancora presente e se ne misura l'inserimento tramite `time.perf_counter()`.
2. Estrarre casualmente una chiave dall'albero e la si rimuove *senza cronometrare* l'operazione.

Questa tecnica garantisce che prima di ogni inserimento (e dunque prima che il cronometro parta), la dimensione torna sempre a essere esattamente n . Inoltre la continua eliminazione e reinserimento di nodi altera in modo casuale la topologia della struttura dati, garantendo che le misurazioni avvengano su albero con forme uniformemente distribuite.

Struttura del Progetto

Il progetto è stato strutturato nel seguente modo, seguendo un'ottica di programmazione ad oggetti:

- **Codici delle Strutture Dati:** I file `BST.py`, `AVL.py` e `RBT.py` contengono i codici completi delle strutture dati. In particolare nei codici delle ultime due strutture dati il codice è stato alleggerito riciclando alcune parti del codice da `BST.py`.
- **KeyManager:**

Misurazione

Per la misurazione abbiamo utilizzato metodi come, per esempio, `perf_counter()` per misurare il tempo di esecuzione dell'inserimento di chiavi in alberi BST, AVL e RB. Abbiamo eseguito più iterazioni per ogni valore di n per ottenere stime robuste dei tempi di esecuzione. Abbiamo anche utilizzato grafici per visualizzare i risultati e confrontare le prestazioni dei diversi tipi di alberi. I grafici mostrano il tempo medio di inserimento in funzione del numero di chiavi presenti nell'albero, evidenziando le differenze tra BST, AVL e RB.

Algoritmo

Il progetto implementa un sistema di misurazione delle prestazioni per tre diverse strutture dati ad albero: Alberi Binari di Ricerca (BST), Alberi AVL e Alberi Rosso-Nero (RBT). L'obiettivo è quantificare il tempo di inserimento di una chiave in ciascun tipo di albero al variare della dimensione dell'albero stesso.

Struttura del Progetto

Il progetto è organizzato nei seguenti moduli:

- **ExperimentSetup:** Configura i parametri dell'esperimento, calcolando una progressione geometrica di valori di n da testare (da 1000 a 10.000.000 elementi) e inizializzando gli alberi con n chiavi casuali.
- **ExperimentRunner:** Esegue una serie di esperimenti controllati per ciascun tipo di albero e per ogni valore di n .
- **BST, AVL, RBT:** Implementano le tre strutture dati con operazioni di inserimento, ricerca e rimozione.
- **KeyManager:** Gestisce l'insieme di chiavi utilizzate negli esperimenti.
- **Plotter:** Visualizza i risultati degli esperimenti tramite grafici.

Algoritmo di Misurazione

Per ogni valore di n nella progressione geometrica, l'algoritmo esegue i seguenti passi:

1. Inizializzazione: Viene creato un albero vuoto (BST, AVL o RBT).
2. Popolazione iniziale: L'albero viene popolato con n chiavi casuali generate da una lista predefinita di chiavi.
3. Ciclo di misurazione: Per 10 iterazioni (configurabili):
 - (a) Viene selezionata una chiave disponibile dall'insieme delle chiavi non ancora inserite.
 - (b) Si misura il tempo di inserimento di questa chiave nell'albero utilizzando `time.perf_counter()`.
 - (c) Viene rimossa casualmente una chiave dall'albero per mantenere la dimensione costante e simulare un carico dinamico.
4. Calcolo della mediana: Viene calcolata la mediana dei 10 tempi di inserimento misurati, che rappresenta una stima robusta delle prestazioni.

Caratteristiche Implementative

- **Progressione geometrica:** I valori di n sono generati secondo una progressione geometrica per coprire uniformemente lo spazio logaritmico tra il minimo e il massimo.
- **Bilanciamento dinamico:** AVL e RBT mantengono automaticamente il bilanciamento dell'albero dopo ogni inserimento e rimozione.
- **Gestione della memoria:** Dopo ogni esperimento completo per un valore di n , la memoria viene liberata mediante garbage collection per evitare interferenze tra esperimenti successivi.
- **Raccolta dati robusta:** L'uso della mediana invece della media riduce l'impatto di misurazioni anomale e fornisce stime più affidabili.